THE DIRECTION (GRADIENT)

Suppose you are standing on the top of the error mountain at point A. To get to the bottom, you need to determine the step direction that results in the deepest descent (has the steepest slope). And what is the slope, again? It is the derivative of the curve. So if you are standing on top of that mountain, you need to look at all the directions around you and find out which direction will result in the deepest descent (1, 2, 3, or 4, for example). Let's say it is direction 3; we choose that way. This brings us to point B, and we restart the process (calculate feedforward and error) and find the direction of deepest descent, and so forth, until we get to the bottom of the mountain.

This process is called *gradient descent*. By taking the derivative of the error with respect to the weight ($\frac{dE}{dw}$), we get the direction that we should take. Now there's one thing left. The gradient only determines the direction. How large should the size of the step be? It could be a 1-foot step or a 100-foot jump. This is what we need to determine next.

THE STEP SIZE (LEARNING RATE α)

The *learning rate* is the size of each step the network takes when it descends the error mountain, and it is usually denoted by the Greek letter alpha (α). It is one of the most important hyperparameters that you tune when you train your neural network (more on that later). A larger learning rate means the network will learn faster (since it is descending the mountain with larger steps), and smaller steps mean slower learning. Well, this sounds simple enough. Let's use large learning rates and complete the neural network training in minutes instead of waiting for hours. Right? Not quite. Let's take a look at what could happen if we set a very large learning rate value.

In figure 2.32, you are starting at point A. When you take a large step in the direction of the arrow, instead of descending the error mountain, you end up at point B, on the other side. Then another large step takes you to C, and so forth. The error will keep *oscillating* and will never descend. We will talk more later about tuning the learning rate and how to determine if the error is oscillating. But for now, you need to know this: if you use a very small learning rate, the network will eventually descend the

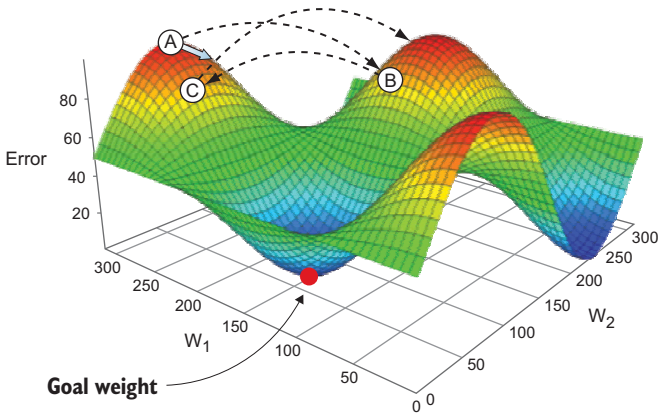

Figure 2.32 Setting a very large learning rate causes the error to oscillate and never descend.